PYTHON UNIT 4

A Python GUI, or Graphical User Interface, is a visual way for users to interact with a software application. GUIs use graphical elements like buttons, checkboxes, and sliders to help users perform tasks and access application features.

Some Python GUI frameworks include:

- **Tkinter**: The standard Python GUI library that comes preinstalled with Python. It's easy to learn and use, making it a good choice for beginners.
- **PyQt**: A cross-platform framework that supports applications on Windows, Mac, Linux, iOS, and Android.
- **Kivy**: An open-source framework that's cross-platform and often used for developing multi-touch applications.
- **wxPython**: Creates native user interfaces without adding extra overhead to the application.
- **PySimpleGUI**: Creates desktop applications with a simpler programming interface.
- **PyGUI**: A cross-platform framework for Unix, Macintosh, and Windows.
  You can use a GUI to create applications for mobile phones, where users can interact with functions by touching, tapping, and swiping on the display.

Python Tkinter

Python Tkinter is a standard GUI (Graphical User Interface) library for Python which provides a fast and easy way to create desktop applications. Tkinter provides a variety of widgets like buttons, labels, text boxes, menus and more that can be used to create interactive user interfaces. Tkinter supports event-driven programming, where actions are taken in response to user events like clicks or keypresses.

Create First Tkinter GUI Application

To create a Tkinter Python app, follow these basic steps:

1. **Import the tkinter module**: Import the tkinter module, which is necessary for creating the GUI components.
2. **Create the main window (container)**: Initialize the main application window using the Tk() class.

3. **Set Window Properties**: We can set properties like the title and size of the window.
4. **Add widgets to the main window**: We can add any number of widgets like buttons, labels, entry fields, etc., to the main window to design the interface.
5. **Pack Widgets:** Use geometry managers like pack(), grid() or place() to arrange the widgets within the window.
6. **Apply event triggers to the widgets**: We can attach event triggers to the widgets to define how they respond to user interactions.

There are two main methods used which the user needs to remember while creating the Python application with GUI.

Tk()

To create a main window in Tkinter, we use the Tk() class. The syntax for creating a main window is as follows:

root = tk.Tk(screenName=None, baseName=None, className='Tk', useTk=1)

- **screenName:** This parameter is used to specify the display name.
- **baseName:** This parameter can be used to set the base name of the application.
- **className:** We can change the name of the window by setting this parameter to the desired name.
- **useTk:** This parameter indicates whether to use Tk or not.

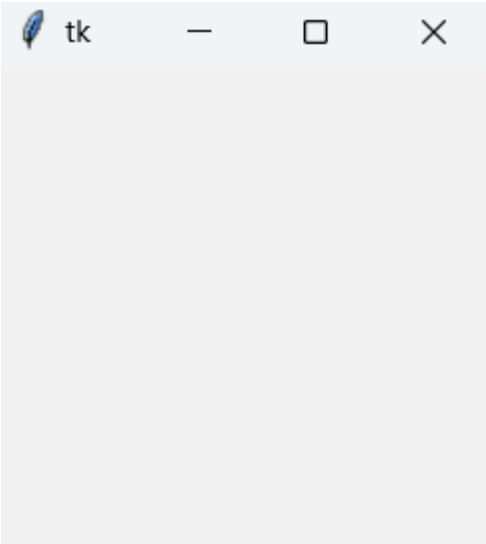
mainloop()

The mainloop() method is used to run application once it is ready. It is an infinite loop that keeps the application running, waits for events to occur (such as button clicks) and processes these events as long as the window is not closed.

Example:

```
import tkinter
m = tkinter.Tk()
'''
widgets are added here
'''
m.mainloop()
```

Output



Tkinter Widget

There are a number of tkinter widgets which we can put in our tkinter application. Some of the major widgets are explained below:

1. Label

It refers to the display box where we display text or image. It can have various options like font, background, foreground, etc. The general syntax is:

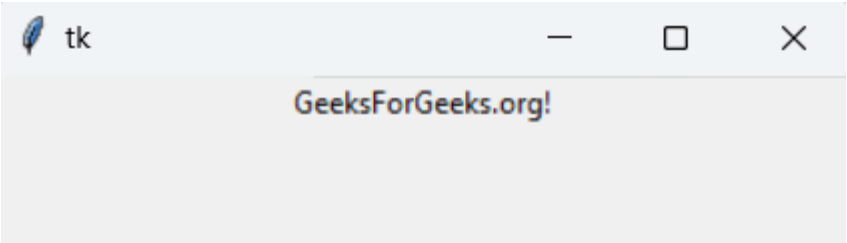
w=Label(master, option=value)

- **master** is the parameter used to represent the parent window.

Example:

```
from tkinter import *
root = Tk()
w = Label(root, text='GeeksForGeeks.org!')
w.pack()
root.mainloop()
```

Output:



Note: We have a number of options and parameters that we can pass to widgets, only some them are used in the examples given in this article.

2. Button

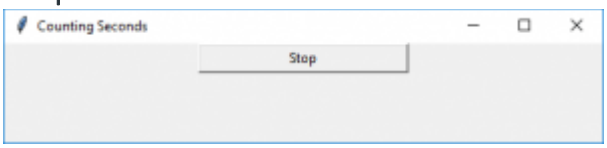
A clickable button that can trigger an action. The general syntax is:

w=Button(master, option=value)

Example:

```
import tkinter as tk

r = tk.Tk()
r.title('Counting Seconds')
button = tk.Button(r, text='Stop', width=25, command=r.destroy)
button.pack()
r.mainloop()
```

Output:



3. Entry

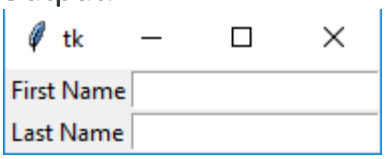
It is used to input the single line text entry from the user. For multi-line text input, Text widget is used. The general syntax is:

w=Entry(master, option=value)

Example:

```
from tkinter import *

master = Tk()
Label(master, text='First Name').grid(row=0)
Label(master, text='Last Name').grid(row=1)
e1 = Entry(master)
e2 = Entry(master)
e1.grid(row=0, column=1)
e2.grid(row=1, column=1)
mainloop()
```

Output:



4. CheckButton

A checkbox can be toggled on or off. It can be linked to a variable to store its state. The general syntax is:

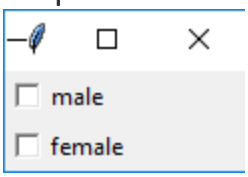
w = CheckButton(master, option=value)

Example:

```
from tkinter import *

master = Tk()
var1 = IntVar()
```

```
Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)
var2 = IntVar()
Checkbutton(master, text='female', variable=var2).grid(row=1, sticky=W)
mainloop()
```

Output:



5. RadioButton

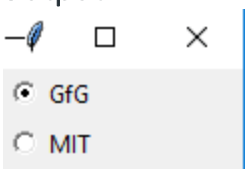
It allows the user to select one option from a set of choices. They are grouped by sharing the same variable. The general syntax is:

w = RadioButton(master, option=value)

Example:

```
from tkinter import *

root = Tk()
v = IntVar()
Radiobutton(root, text='GfG', variable=v, value=1).pack(anchor=W)
Radiobutton(root, text='MIT', variable=v, value=2).pack(anchor=W)
mainloop()
```

Output



6. Listbox

It displays a list of items from which a user can select one or more. The general syntax is:

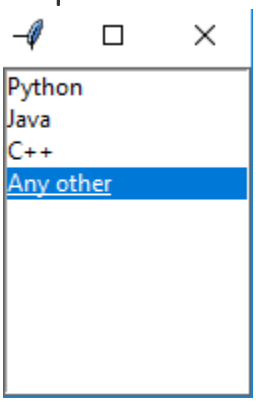
w = Listbox(master, option=value)

Example:

```
from tkinter import *

top = Tk()
Lb = Listbox(top)
Lb.insert(1, 'Python')
Lb.insert(2, 'Java')
Lb.insert(3, 'C++')
Lb.insert(4, 'Any other')
Lb.pack()
```

```
top.mainloop()
```

Output



7. Scrollbar

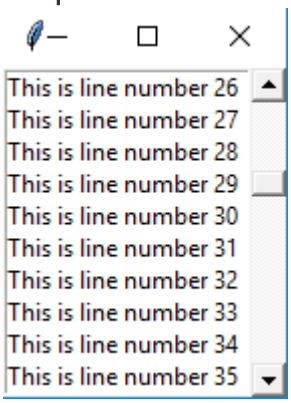
It refers to the slide controller which will be used to implement listed widgets. The general syntax is:

w = Scrollbar(master, option=value)

Example:

```
from tkinter import *

root = Tk()
scrollbar = Scrollbar(root)
scrollbar.pack(side=RIGHT, fill=Y)
mylist = Listbox(root, yscrollcommand=scrollbar.set)

for line in range(100):
    mylist.insert(END, 'This is line number' + str(line))

mylist.pack(side=LEFT, fill=BOTH)
scrollbar.config(command=mylist.yview)
mainloop()
```

Output



8. Menu

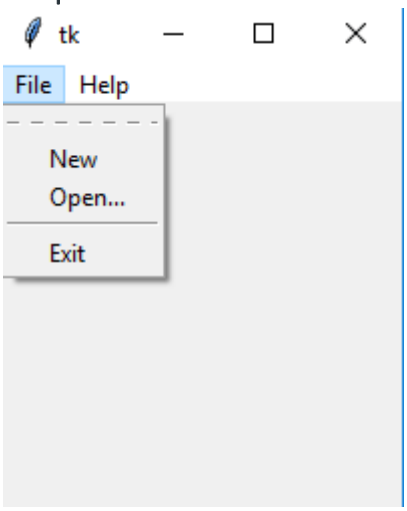
It is used to create all kinds of menus used by the application. The general syntax is:

window.w = Menu(master, option=value)

Example:

```
from tkinter import *

root = Tk()
menu = Menu(root)
root.config(menu=menu)
filemenu = Menu(menu)
menu.add_cascade(label='File', menu=filemenu)
filemenu.add_command(label='New')
filemenu.add_command(label='Open...')
filemenu.add_separator()
filemenu.add_command(label='Exit', command=root.quit)
helpmenu = Menu(menu)
menu.add_cascade(label='Help', menu=helpmenu)
helpmenu.add_command(label='About')
mainloop()
```

Output:



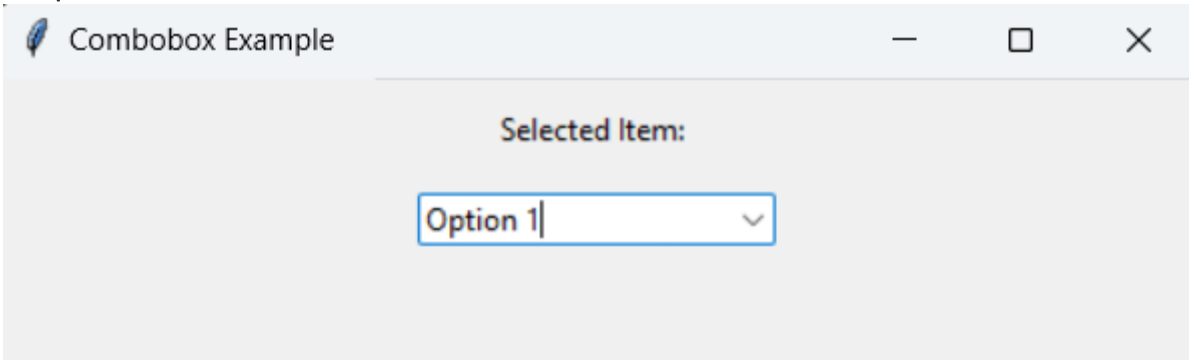
9. Combobox

Combobox widget is created using the ttk.Combobox class from the tkinter.ttk module. The values for the Combobox are specified using the values parameter. The default value is set using the set method. An event handler function on_select is bound to the Combobox using the bind method, which updates a label with the selected item whenever an item is selected.

```
import tkinter as tk
from tkinter import ttk
```

```
def select(event):
    selected_item = combo_box.get()
    label.config(text="Selected Item: " + selected_item)

root = tk.Tk()
root.title("Combobox Example")

# Create a label
label = tk.Label(root, text="Selected Item: ")
label.pack(pady=10)

# Create a Combobox widget
combo_box = ttk.Combobox(root, values=["Option 1", "Option 2", "Option 3"])
combo_box.pack(pady=5)

# Set default value
combo_box.set("Option 1")

# Bind event to selection
combo_box.bind("<<ComboboxSelected>>", select)

root.mainloop()
```

Output:



10. Scale

It is used to provide a graphical slider that allows to select any value from that scale. The general syntax is:

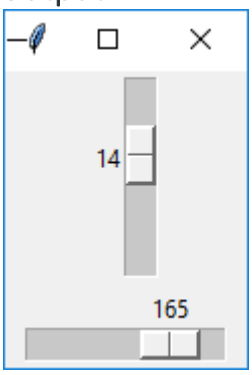
w = Scale(master, option=value)

Example:

```
from tkinter import *

master = Tk()
```

```
w = Scale(master, from_=0, to=42)
w.pack()
w = Scale(master, from_=0, to=200, orient=HORIZONTAL)
w.pack()
mainloop()
```

Output:



How to install MySQL connector package in Python

•

MySQL is a Relational Database Management System (RDBMS) whereas the structured Query Language (SQL) is the language used for handling the RDBMS using commands i.e Creating, Inserting, Updating and Deleting the data from the databases.

A connector is employed when we have to use MySQL with other programming languages. The work of mysql-connector is to provide access to MySQL Driver to the required language. Thus, it generates a connection between the programming language and the MySQL Server.

Installation:

To install Python-mysql-connector module, one must have **Python** and **PIP**, preinstalled on their system. To check if your system already contains Python, go through the following instructions:

Open the Command line(search for cmd in the Run dialog(**+ R**).

Now run the following command:

```
python --version
```

If Python is already installed, it will generate a message with the Python version available.

```
C:\Windows\system32\cmd.exe

C:\Users\Abhinav Singh>python --version
Python 3.8.1

C:\Users\Abhinav Singh>
```

If Python is not present, go through How to install Python on Windows and Linux? and follow the instructions provided.

PIP is a package management system used to install and manage software packages/libraries written in Python. These files are stored in a large "on-line repository" termed as Python Package Index (PyPI).
To check if PIP is already installed on your system, just go to the command line and execute the following command:

```
pip -V
```

```
C:\Windows\system32\cmd.exe

C:\Users\Abhinav Singh\AppData\Local\Programs\Python\Python38-32>pip -V
pip 19.3.1 from c:\users\abhinav singh\appdata\local\programs\python\python38-32\lib\site-packages\pip (python 3.8)

C:\Users\Abhinav Singh\AppData\Local\Programs\Python\Python38-32>
```

If PIP is not present, go through How to install PIP on Windows and Linux?

Windows

mysql-connector method can be installed on Windows with the use of following command:

```
pip install mysql-connector-python
```

```
Command Prompt

C:\Users\Abhinav>pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading https://files.pythonhosted.org/packages/5c/1e/3f372b31853b868153e453146d99ca787da3eb4bf0b65
4590b829b262afa/mysql_connector_python-8.0.19-py2.py3-none-any.whl (355kB)
     |                                | 358kB 233kB/s
Collecting dnspython==1.16.0 (from mysql-connector-python)
  Downloading https://files.pythonhosted.org/packages/ec/d3/3aa0e7213ef72b8585747aa0e271a9523e713813b9a20
177ebe1e939deb0/dnspython-1.16.0-py2.py3-none-any.whl (188kB)
     |                                | 194kB 2.2MB/s
Collecting protobuf==3.6.1 (from mysql-connector-python)
  Downloading https://files.pythonhosted.org/packages/77/78/a7f1ce761e2c738e209857175cd4f90a8562d1bde3286
8a8cd5290d58926/protobuf-3.6.1-py2.py3-none-any.whl (390kB)
     |                                | 399kB 3.3MB/s
Collecting six>=1.9 (from protobuf==3.6.1->mysql-connector-python)
  Downloading https://files.pythonhosted.org/packages/65/eb/1f97cb97bfc2390a276969c6fae16075da282f5058082
d4cb10c6c5c1dba/six-1.14.0-py2.py3-none-any.whl
Requirement already satisfied: setuptools in c:\program files\windowsapps\pythonsoftwarefoundation.python
.3.8_3.8.752.0_x64__qbz5n2kfra8p0\lib\site-packages (from protobuf==3.6.1->mysql-connector-python) (41.2.
0)
Installing collected packages: dnspython, six, protobuf, mysql-connector-python
Successfully installed dnspython-1.16.0 mysql-connector-python-8.0.19 protobuf-3.6.1 six-1.14.0
WARNING: You are using pip version 19.2.3, however version 20.0.2 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\Users\Abhinav>
```

Linux

mysql-connector method can be installed on Linux with the use of following command:

```
pip3 install mysql-connector
```

```
nikhil@nikhil-Lenovo-V130-15IKB:~$ pip3 install mysql-connector
Collecting mysql-connector
Installing collected packages: mysql-connector
Successfully installed mysql-connector-2.2.9
nikhil@nikhil-Lenovo-V130-15IKB:~$
```

Connect MySQL database using MySQL-Connector Python

Last Updated : 18 Mar, 2023

- While working with Python we need to work with databases, they may be of different types like MySQL, SQLite, NoSQL, etc. In this article, we will be looking forward to how to connect MySQL databases using MySQL Connector/Python. MySQL Connector module of Python is used to connect MySQL databases with the Python programs, it does that using the Python Database API Specification v2.0 (PEP 249). It uses the Python standard library and has no dependencies.

Connecting to the Database

The mysql.connector provides the connect() method used to create a connection between the MySQL database and the Python application. The syntax is given below.

Syntax:

```
Conn_obj= mysql.connector.connect(host = <hostname>, user = <username>, passwd = <password>)
```

The **connect()** function accepts the following arguments.
Hostname – It represents the server name or IP address on which MySQL is running.
Username – It represents the name of the user that we use to work with the MySQL server. By default, the username for the MySQL database is root.
Password – The password is provided at the time of installing the MySQL database. We don't need to pass a password if we are using the root.
Database – It specifies the database name which we want to connect. This argument is used when we have multiple databases.

In the following example we will be connecting to MySQL database using connect()
Example:

- Python3

```
# Python program to connect

# to mysql database




import mysql.connector
```

```
# Connecting from the server

conn = mysql.connector.connect(user = 'username',

                               host = 'localhost',

                              database = 'database_name')



print(conn)



# Disconnecting from the server

conn.close()
```

Output:

```
<mysql.connector.connection_cext.CMySQLConnection object at 0x7f55f10ce0b8>
```

Also for the same, we can use connection.MySQLConnection() class instead of connect():

Example:

- Python3

```
# Python program to connect

# to mysql database
```

```
from mysql.connector import connection


# Connecting to the server

conn = connection.MySQLConnection(user = 'username',

                                  host = 'localhost',

                                  database = 'database_name')


print(conn)


# Disconnecting from the server

conn.close()
```

Output:

```
<mysql.connector.connection.MySQLConnection object at 0x7f55e47a6358>
```

Another way is to pass the dictionary in the connect() function using '**' operator:

Example:

- Python3

```
# Python program to connect

# to mysql database




from mysql.connector import connection




dict = {

  'user': 'root',

  'host': 'localhost',

  'database': 'College'

}



# Connecting to the server

conn = connection.MySQLConnection(**dict)



print(conn)
```

```
# Disconnecting from the server

conn.close()
```

Output:

```
<mysql.connector.connection.MySQLConnection object at 0x7f55e47a6630>
```

Example

- Python3

```
# importing required libraries

import mysql.connector



dataBase = mysql.connector.connect(

host ="localhost",

user ="user",

passwd ="gfg"

)



# preparing a cursor object

cursorObject = dataBase.cursor()
```

```
# creating database

cursorObject.execute("CREATE DATABASE geeks4geeks")
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| College            |
| geeks4geeks        |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.01 sec)
```